

Deployment Guide 🚀

This guide covers various ways to deploy the Senpai Waifu Bot.

Table of Contents

- Local Development
- Docker Deployment
- Heroku Deployment
- VPS Deployment
- Railway Deployment

Local Development

Prerequisites

- Go 1.21+
- MongoDB database

Steps

1. Clone the repository
2. Copy `.env.example` to `.env` and fill in your values
3. Run `go mod download`
4. Run `go run ./cmd/bot`

Docker Deployment

Using Docker Compose (Recommended)

1. Create `.env` file with your configuration
2. Run:

```
docker-compose up -d
```

3. View logs:

```
docker-compose logs -f
```

4. Stop the bot:

```
docker-compose down
```

Using Docker directly

1. Build the image:

```
docker build -t senpai-waifu-bot .
```

2. Run the container:

```
docker run -d \  
  
--name senpai-bot \  
  
--env-file .env \  
  
-e TZ=Asia/Kolkata \  
  
senpai-waifu-bot
```

Heroku Deployment

Using Heroku CLI

1. Login to Heroku:

```
heroku login
```

2. Create a new app:

```
heroku create your-bot-name
```

3. Set environment variables:

```
heroku config:set BOT_TOKEN=your_token  
heroku config:set BOT_USERNAME=your_bot_username  
heroku config:set API_ID=your_api_id  
heroku config:set API_HASH=your_api_hash  
heroku config:set OWNER_ID=your_owner_id  
heroku config:set GROUP_ID=your_group_id  
heroku config:set CHARA_CHANNEL_ID=your_channel_id  
heroku config:set MONGO_URL=your_mongo_url
```

4. Deploy:

```
git push heroku main
```

5. Scale the worker:

```
heroku ps:scale worker=1
```

Using Heroku Dashboard

1. Create a new app on Heroku
2. Connect your GitHub repository
3. Enable automatic deploys (optional)
4. Add all environment variables in Settings > Config Vars
5. Deploy the app

VPS Deployment

Using Systemd (Ubuntu/Debian)

1. Build the bot:

```
go build -o bot ./cmd/bot
```

2. Create systemd service file:

```
sudo nano /etc/systemd/system/senpai-bot.service
```

3. Add the following content:

```
[Unit]
Description=Senpai Waifu Bot
After=network.target

[Service]
Type=simple
User=your_username
WorkingDirectory=/path/to/bot
Environment="BOT_TOKEN=your_token"
Environment="MONGO_URL=your_mongo_url"

# Add other environment variables

ExecStart=/path/to/bot/bot
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

4. Enable and start the service:

```
sudo systemctl daemon-reload
sudo systemctl enable senpai-bot
sudo systemctl start senpai-bot
```

5. Check status:

```
sudo systemctl status senpai-bot
```

6. View logs:

```
sudo journalctl -u senpai-bot -f
```

Using PM2 (Node.js process manager)

1. Install PM2:

```
npm install -g pm2
```

2. Create ecosystem file:

```
pm2 init
```

3. Edit ecosystem.config.js:

```
module.exports = {  
  apps: [{  
    name: 'senpai-bot',  
    script: './bot',  
    env: {  
      BOT_TOKEN: 'your_token',  
      MONGO_URL: 'your_mongo_url',  
      // Add other env vars  
    },  
    autorestart: true,  
    max_restarts: 10,  
    min_uptime: '10s'  
  ]  
};
```

4. Start with PM2:

```
pm2 start ecosystem.config.js
```

5. Save PM2 config:

```
pm2 save  
pm2 startup
```

Railway Deployment

1. Create a Railway account at <https://railway.app>
2. Create a new project
3. Connect your GitHub repository
4. Add environment variables in Variables section
5. Deploy the project

Environment Variables

Make sure to set all required environment variables:

	Variable	Description	Required
	BOT_TOKEN	Telegram bot token from @BotFather	Yes
	BOT_USERNAME	Your bot's username	Yes
	API_ID	Telegram API ID	Yes
	API_HASH	Telegram API Hash	Yes
	OWNER_ID	Your Telegram user ID	Yes
	GROUP_ID	Main group ID	Yes
	CHARA_CHANNEL_ID	Character channel ID	Yes
	MONGO_URL	MongoDB connection string	Yes
	SUDO_USERS	Comma-separated sudo user IDs	No
	VIDEO_URL	Comma-separated video URLs	No
	SUPPORT_CHAT	Support chat username	No
	UPDATE_CHAT	Update channel username	No

Monitoring

Health Check

The bot automatically responds to `/ping` command to check latency.

Logs

- Docker: `docker-compose logs -f`
- Systemd: `sudo journalctl -u senpai-bot -f`
- PM2: `pm2 logs senpai-bot`

Troubleshooting

Bot not responding

1. Check if bot token is correct
2. Verify MongoDB connection
3. Check logs for errors

High latency

1. Check server location (should be close to Telegram servers)
2. Verify MongoDB connection speed
3. Check for network issues

Database errors

1. Verify MongoDB connection string
2. Check if database is accessible
3. Ensure proper permissions

Performance Tips

1. Use a VPS close to Telegram servers (EU recommended)
2. Use MongoDB Atlas for managed database
3. Enable connection pooling
4. Use SSD storage for better I/O performance